

SECURITY

# The First 72 Hours

*An incident response field manual for systems that call a model.*

IN THIS PAPER

01 Three things that are different
02 Hours 0 to 4: contain
03 Hours 4 to 24: scope
04 Hours 24 to 72: restore and communicate
05 A 90-minute tabletop that finds the gaps

October 2026 CloudyAI Research Free to download - no email required

SUMMARY

*AI incidents break the assumptions of classic response: non-deterministic behaviour, third-party model versions, and evidence spread across prompt logs. A phase-by-phase manual with the evidence sources and rollback decisions that matter.*

The call usually starts the same way: something the system said or did was wrong in public. A support agent promised a refund the business does not offer. A retrieval assistant surfaced a document it should never have indexed. An agent with a write tool deleted something it was not meant to touch. An API key found its way into a client-side bundle.

AI incidents are still security incidents. Containment, evidence, communication, and prevention all apply. But three things break the assumptions of a classic playbook, and teams that ignore them spend the first day collecting the wrong evidence.

## Three things that are different

— Behaviour is not deterministic. The same input can produce a different output, so 'reproduce the bug' is often not possible in the way a software team expects. You need the recorded input, the model version, and the prompt version together.

— Part of the system belongs to someone else. The model is a third-party dependency that can change under you. Provider audit logs, model version IDs, and rate limits are part of your evidence, and your provider's support channel is part of your response.

— The evidence is semantic. There is no packet capture of a model's reasoning. What you have is a log of prompts, responses, retrieved context, and tool calls - and if you did not log those, the first 24 hours become an exercise in archaeology.

> **The preparation that decides the incident**
>
> Pin model versions. Version your prompts in git. Log prompt, response, retrieved document IDs, model version, and tool calls per request. Keep a kill switch for tools with side effects. Everything else in this paper is easier if those four things exist before you need them.

## Hours 0 to 4: contain

01 Name an incident lead and start a single timeline document. Times in UTC, decisions and who made them. One document beats four chat threads.

02 Stop the bleeding with the smallest lever that works: revoke the key, disable the tool, roll back the prompt, pin the previous model version, or put the feature behind a flag. Prefer a reversible lever over a clever one.

03 Preserve before you clean. Export prompt and response logs, model version IDs, the deployed prompt text, retrieval index snapshot or document ID list, feature flag state, and provider audit logs. Cleaning up evidence is the most common irreversible mistake.

04 Determine whether personal or regulated data left the boundary. If yes, the clock on notification has started and legal counsel joins now, not later.

05 Brief one internal audience - the people who need to act - with three facts: what we know, what we have done, what happens next. Do not brief the whole company in hour one.

# Hours 4 to 24: scope

### Build the incident's shape

How many requests are affected? Over what window? Which users, tenants, or regions? Was the behaviour caused by a prompt change, a model update, a poisoned document, an injected input, or a user doing something the design never anticipated? Each answer leads to a different fix, and guessing produces a second incident.

| Evidence source | What it tells you | Where it usually lives |
|---|---|---|
| Gateway / proxy logs | Every prompt and response, with latency and status | API gateway or LLM proxy |
| Model + version IDs | Whether behaviour changed under you | Request metadata, provider console |
| Prompt and system text | What the system was told to do at the time | Git history, config store |
| Retrieval snapshots | What documents were in scope | Vector store versioning or document list |
| Tool-call logs | What the agent actually did, with arguments | Application logs, audit trail |
| Identity and access logs | Who or what invoked the system | IAM, SSO, API key audit |
| Provider audit logs | Requests from your account, including unknown keys | Provider console |

### The rollback decision

Roll back first and investigate second when the harm is ongoing, the cause is unclear, and the rollback is cheap. Investigate first only when the rollback itself creates a larger risk - for example, disabling a safety alerting system. Write the decision and the reason in the timeline, even when it turns out to be wrong.

# Hours 24 to 72: restore and communicate

Restoration is not 'turn it back on'. It is: fix the cause, add the guard that would have caught it, verify on a replay of the incident inputs, and re-enable gradually with a watch. If the verification step is skipped, the same incident returns with a different date.

Communication should be drafted by a human and reviewed by whoever owns legal and customer trust. Three audiences usually matter: affected customers, the regulator where the law requires notification, and your own team. Write plainly, state what you know and what you do not, and avoid explaining model behaviour you have not verified.

**On regulator clocks**

Many regimes require notification of certain personal data breaches within 72 hours of awareness - GDPR is the familiar example. Zimbabwe's Cyber and Data Protection Act requires notification to the supervising authority in defined circumstances, and the specifics depend on the facts. Confirm obligations with counsel early; do not let an engineering investigation consume the notification window.

## A 90-minute tabletop that finds the gaps

Run one of these before you need it. Five scenarios, fifteen minutes each, no slides.

- A support agent promises a 40 per cent discount to a customer, in writing, on a recorded channel.
- An API key appears in a public repository and has been used from an unfamiliar address.
- A retrieved document contains instructions that cause the assistant to ignore its system prompt.
- An agent with a ticketing integration closes the wrong 200 tickets overnight.
- The provider silently updates the model and your extraction accuracy drops by a third.

Score each scenario on four questions: could we detect it, could we stop it, could we prove what happened, and could we tell the right people in time? The answers are your roadmap, and they are usually uncomfortable in a useful way.

## The short checklist

| Phase | Non-negotiables |
|---|---|
| Before | Pinned model versions; prompts in git; per-request logging; tool kill switch; named on-call; provider contacts |
| 0-4 h | Incident lead; timeline; smallest reversible lever; evidence preserved; data-egress assessment |
| 4-24 h | Scope by request, user, and window; cause hypothesis tested against logs; rollback decision recorded |
| 24-72 h | Root cause fixed; guard added; replay verification; gradual re-enable; notifications drafted and sent |
| After | Blameless review within two weeks; two concrete changes; tabletop scheduled |

## The uncomfortable conclusion

Most organisations will have an AI incident before they have an AI incident process. The gap is not technical maturity; it is the decision to write down, in advance, what you will do when the system does something nobody intended. That decision costs an afternoon and saves a week.

## ABOUT THIS PAPER

CloudyAI is a Zimbabwean AI studio. We deploy computer vision on existing plant cameras, audit AI spend, and help teams respond when an AI system misbehaves. Research is published free, without an email wall.

Corrections and questions: research@cloudyai.co.zw. If you would like this method applied to your plant, the first conversation is free and the camera audit is a fixed-price week.